

UT6. Programación en Bases de Datos



Índice de contenidos – UT6: Programación en MySQL

1. Introducción
2. Conceptos básicos
 - 2.1. Unidades léxicas (I)
 - 2.1.1. Unidades léxicas (II)
 - 2.2. Tipos de datos simples, variables y constantes
 - 2.2.1. Subtipos
 - 2.2.2. Variables y constantes
 - 2.3. El bloque en MySQL
 - 2.4. Estructuras de control (I)
 - 2.4.1. Estructuras de control (II). Bucles
 - 2.5. Manejo de errores (I)
 - 2.5.1. Manejo de errores (II)
 - 2.5.2. Manejo de errores (III)
 - 2.5.3. Manejo de errores (IV)
 - 2.6. Sentencias SQL en programas MySQL
3. Tipos de datos compuestos
 - 3.1. Registros
 - 3.2. Colecciones: Arrays de longitud variable
 - 3.2.1. Tablas anidadas
 - 3.3. Cursores
 - 3.3.1. Cursores explícitos
 - 3.3.2. Cursores variables
4. Abstracción en MySQL
 - 4.1. Subprogramas
 - 4.1.1. Almacenar subprogramas en la base de datos
 - 4.1.2. Parámetros de los subprogramas
 - 4.1.3. Sobrecarga y recursividad
 - 4.2. Paquetes
5. Disparadores
 - 5.1. Definición de disparadores de tablas
 - 5.2. Ejemplos de disparadores

1. Introducción

En el desarrollo de aplicaciones reales, es frecuente que se requiera programar determinadas tareas directamente en el sistema gestor de bases de datos (SGBD). MySQL, como uno de los sistemas gestores más utilizados, ofrece un conjunto de herramientas que permiten definir lógica de negocio directamente dentro del propio sistema de base de datos.

Esta unidad tiene como objetivo enseñar cómo estructurar y codificar dicha lógica utilizando el lenguaje de procedimientos que proporciona MySQL. Se tratará el uso de variables, estructuras de control, cursores, subprogramas y mecanismos de control de errores, entre otros, centrándonos en los mecanismos que proporciona MySQL a través de su extensión SQL/PSM (Persistent Stored Modules).

La programación en bases de datos permite encapsular procesos recurrentes, asegurar la coherencia de los datos, mejorar el rendimiento y simplificar la lógica de las aplicaciones cliente.

A lo largo de esta unidad se utilizará una base de datos de ejemplo que permitirá ilustrar todos los conceptos mediante ejemplos prácticos, incluyendo scripts que podrán ejecutarse directamente en MySQL para facilitar la experimentación y el aprendizaje.

2. Conceptos básicos

Caso práctico

Juan ha avanzado muy rápido en la lectura del manual. Como ya sabe programar en otros lenguajes, le resulta sencillo entender los tipos de datos, estructuras de control y reglas sintácticas. Decide ayudar a **María**, que está empezando en programación, para que pronto puedan trabajar juntos en la base de datos de juegos on-line.

En este apartado se introducen los conceptos esenciales para programar en **MySQL**:

- Reglas sintácticas
- Elementos básicos del lenguaje
- Tipos de datos disponibles
- Estructuras de control (condicionales e iterativas)
- Manejo de errores

-  Permite reducir el tráfico en la red en entornos cliente-servidor.
-  No podemos utilizar sentencias SQL dentro de un bloque.
-  Nos ofrece las ventajas de SQL y la potencia de un lenguaje procedimental.
-  Para utilizar procedimientos almacenados debemos instalar drivers adicionales.

2.1. Unidades léxicas (I)

MySQL no distingue entre mayúsculas y minúsculas en palabras clave (aunque sí distingue en los literales de cadena). Las unidades léxicas incluyen:

- Delimitadores
- Identificadores
- Literales
- Comentarios

Delimitadores

Delimitador	Significado	Delimitador compuesto	Significado
+	Suma	!= o <>	Distinto
-	Resta o negación	<=	Menor o igual
*	Producto	>=	Mayor o igual
/	División	:=	Asignación ( no válido en MySQL)
=	Igual relacional		Concatenación de cadenas
;	Fin de instrucción		

 **Consulta adicional:** [MySQL Lexical Structure](#)

2.1.1. Unidades léxicas (II)

Identificadores

Deben comenzar con una letra o un guion bajo (_).

Pueden contener letras, dígitos, el símbolo \$ y el guion bajo _.

No pueden coincidir con palabras reservadas (a menos que se encierren entre comillas invertidas `).

Ejemplos válidos:

x, codigo_postal, _usuario, monto_total\$

Ejemplos inválidos:

rock&roll, on/off, 123variable, select (si no se usa `select`)

👉 Para usar palabras reservadas como identificadores:

```
SELECT `select`, `from` FROM datos;
```

Literales

- **Numéricos:** 123, 7.45, 2e3
- **Cadenas:** 'texto'
- **Booleanos:** TRUE, FALSE
- **Nulos:** NULL

Comentarios

Una línea:

```
-- Comentario de una línea  
# También válido como comentario de una línea
```

Varias líneas:

```
/* Comentario  
   en varias líneas */
```

📖 Referencia: [MySQL Comments](#)

Variables de usuario (@)

En MySQL podemos declarar variables de sesión utilizando el prefijo @. Este tipo de variables **no necesita declaración previa** y es útil **fuera de procedimientos**, por ejemplo, en consola o Workbench.

- Persisten durante la sesión
- No necesitan tipo declarado
- Son prácticas para pruebas interactivas
- **No** se deben usar dentro de procedimientos (se debe usar DECLARE)

✅ Ejemplo interactivo (en Workbench)

```
SET @precio = 15.50;  
SET @cantidad = 3;
```

```
SET @total = @precio * @cantidad;  
SELECT @total AS total_calculado;
```

✗ En procedimientos

```
-- INCORRECTO  
SET @total = ...;  
  
-- CORRECTO dentro de BEGIN...END:  
DECLARE total DECIMAL(10,2);  
SET total = precio * cantidad;
```

Contexto de uso de @variable

Contexto	¿Usar @variable?
Consola / Workbench	✓ Sí
Procedimientos almacenados	✗ No

Ejercicio resuelto

```
IF A <> B THEN iguales := FALSE; --No son iguales
```

- **Palabras clave:** IF, THEN
- **Identificadores:** A, B, iguales
- **Delimitadores:** <>, :=, ;
- **Literal:** FALSE
- **Comentario:** --No son iguales

Ejercicio propuesto

```
SET total = precio * cantidad; --Cálculo del importe total
```

- **Palabra clave:** SET
- **Identificadores:** total, precio, cantidad
- **Delimitadores:** =, *, ;
- **Comentario:** --Cálculo del importe total

2.2.1. Subtipos

MySQL no permite SUBTYPE como PL/SQL, pero puede simularse con:

```
DECLARE apodo_aux VARCHAR(20);  
DECLARE apodo LIKE apodo_aux;
```

■ Referencia: [MySQL CREATE PROCEDURE](#)

2.2.2. Variables y constantes

- Variables: `DECLARE x INT DEFAULT 5;`
- No existe `CONSTANT`, se simula con declaración sin modificación
- `NOT NULL` exige asignación inmediata
- Conversión de tipos: `CAST(... AS tipo)`, `CONVERT(...)`

Precedencia de operadores

Operador	Operación
!, -, ~	Negaciones
*, /, %	Multiplicación/división
+, -	Suma, resta
=, < >, etc.	Comparaciones
AND, OR	Operadores lógicos

■ Consulta: [Operator precedence – MySQL](#)

Autoevaluación

- $5 + 3 * 2 ** 2 = 17$
- $2 ** 3 + 6 / 3 = 10$
- $2 ** (3 + 6 / 3) = 64$

2.3. El bloque en MySQL

Un bloque en MySQL se estructura mediante la palabra clave `BEGIN` y finaliza con `END`. A diferencia de PL/SQL, no existe una zona explícita `DECLARE` fuera de procedimientos o funciones. Para su definición completa, se utiliza dentro de `CREATE PROCEDURE` o `CREATE FUNCTION`.

Estructura general de un bloque en MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE ejemplo()
```

```
BEGIN
```

```
    -- Declaración de variables
```

```
    DECLARE aux INT DEFAULT 10;
```

```
    -- Sentencias ejecutables
```

```
    IF aux = 10 THEN
```

```
        SELECT 'Correcto';
```

```
    END IF;
```

```
    -- (MySQL gestiona errores de forma limitada con HANDLER)
```

```
END//
```

```
DELIMITER ;
```

Anidamiento de bloques y visibilidad

En MySQL se permite anidar bloques dentro de procedimientos. El ámbito de las variables se limita al bloque en que se declaran.

```
DELIMITER //
```

```
CREATE PROCEDURE pruebaBloque()
```

```
BEGIN
```

```
    DECLARE aux INT DEFAULT 10; -- Variable global
```

```
    BEGIN
```

```
        DECLARE aux INT DEFAULT 5; -- Variable local
```

```
        IF aux = 10 THEN
```

```
            SELECT 'Igual a 10';
```

```
        ELSE
```

```
            SELECT 'No es 10';
```

```
        END IF;
```

```
    END;
```

```
END//
```

```
DELIMITER ;
```

■ **Consulta:** [MySQL – Declaración de variables y bloques](#)

Autoevaluación:

En MySQL los bloques pueden anidarse y cada uno tiene su propio ámbito de visibilidad de variables.

-  Verdadero
-  Falso

2.4. Estructuras de control (I)

Las estructuras de control en **MySQL** permiten alterar el flujo de ejecución del programa. Son similares a las de otros lenguajes procedurales y se dividen en:

- **Condicionales**
- **Iterativas** (veremos más adelante)

Condicionales

IF - THEN

```
IF condicion THEN
    -- sentencias
END IF;
```

IF - THEN - ELSE

```
IF condicion THEN
    -- sentencias si verdadero
ELSE
    -- sentencias si falso
END IF;
```

IF - THEN - ELSIF - ELSE

```
IF condicion1 THEN
    -- bloque1
ELSEIF condicion2 THEN
    -- bloque2
ELSE
    -- bloque por defecto
END IF;
```

Ejemplo en MySQL

```
DELIMITER //
CREATE PROCEDURE compararValores()
```

```
BEGIN
    DECLARE a INT DEFAULT 10;
    DECLARE b INT DEFAULT 7;

    IF a > b THEN
        SELECT CONCAT(a, ' es mayor');
    ELSE
        SELECT CONCAT(b, ' es mayor o iguales');
    END IF;
END;
//
DELIMITER ;
```

CASE

La instrucción CASE permite evaluar múltiples condiciones. Existen dos formas:

CASE con selector

```
CASE selector
    WHEN valor1 THEN ...
    WHEN valor2 THEN ...
    ELSE ...
END CASE;
```

CASE con condiciones de búsqueda

```
CASE
    WHEN condicion1 THEN ...
    WHEN condicion2 THEN ...
    ELSE ...
END CASE;
```

Ejemplo

```
DELIMITER //
CREATE PROCEDURE evaluarNota()
BEGIN
    DECLARE nota INT DEFAULT 8;

    CASE
        WHEN nota IN (1,2) THEN SELECT 'Muy deficiente';
        WHEN nota IN (3,4) THEN SELECT 'Insuficiente';
```

```

        WHEN nota = 5 THEN SELECT 'Suficiente';
        WHEN nota = 6 THEN SELECT 'Bien';
        WHEN nota IN (7,8) THEN SELECT 'Notable';
        WHEN nota IN (9,10) THEN SELECT 'Sobresaliente';
        ELSE SELECT 'Nota no válida';
    END CASE;
END;
//
DELIMITER ;

```

 **Consulta adicional:** [MySQL CASE Syntax](#)

Autoevaluación

¿Existen en MySQL estructuras que permitan tomar una decisión condicional en tiempo de ejecución?

- ☒ Verdadero
- ☐ Falso

2.4.1. Estructuras de control (II). Bucles

Las estructuras de control iterativo permiten ejecutar un conjunto de instrucciones repetidamente mientras se cumpla una condición o durante un número determinado de veces.

LOOP

Bucle sin condición de parada explícita. Necesita una sentencia LEAVE para salir del bucle.

```

-- ☒ No válido directamente en MySQL fuera de BEGIN...END etiquetado
LOOP
    -- instrucciones
END LOOP;

```

En MySQL usamos **etiquetas** y LEAVE para emular esta estructura:

```

DELIMITER //
CREATE PROCEDURE bucle_loop()
BEGIN
    DECLARE a INT DEFAULT 1;

```

```

    etiqueta_bucle: LOOP
        SELECT a;
        IF a > 9 THEN
            LEAVE etiqueta_bucle;
        END IF;
        SET a = a + 1;
    END LOOP etiqueta_bucle;
END;
//
DELIMITER ;

```

LOOP con LEAVE al cumplirse condición (equivalente a EXIT WHEN)

```

DELIMITER //
CREATE PROCEDURE bucle_exit_when()
BEGIN
    DECLARE a INT DEFAULT 1;
    etiqueta: LOOP
        SELECT a;
        IF a > 9 THEN
            LEAVE etiqueta;
        END IF;
        SET a = a + 1;
    END LOOP etiqueta;
END;
//
DELIMITER ;

```

WHILE

Repite el bloque mientras se cumpla una condición.

```

DELIMITER //
CREATE PROCEDURE bucle_while()
BEGIN
    DECLARE a INT DEFAULT 1;
    WHILE a < 10 DO
        SELECT a;
        SET a = a + 1;
    END WHILE;
END;
//
DELIMITER ;

```

FOR

MySQL no dispone de una construcción FOR como PL/SQL, pero puede simularse con WHILE:

```
DELIMITER //
CREATE PROCEDURE bucle_for()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 10 DO
        SELECT i;
        SET i = i + 1;
    END WHILE;

    SET i = 10;
    WHILE i >= 1 DO
        SELECT i;
        SET i = i - 1;
    END WHILE;
END;
//
DELIMITER ;
```

■ **Consulta:** [MySQL - Flow Control](#)

Autoevaluación

Al utilizar REVERSE en un bucle FOR, en PL/SQL el rango debe ser desde el mayor al menor. ¿Se puede hacer lo mismo en MySQL directamente con FOR?

-  Verdadero
-  Falso

Nota: En MySQL se debe simular con WHILE, ya que no tiene FOR nativo.

2.5. Manejo de errores

El manejo de errores en MySQL se realiza mediante bloques de código que permiten capturar y gestionar excepciones. A diferencia de PL/SQL, MySQL no tiene un sistema de excepciones tan robusto, pero ofrece herramientas para manejar errores de forma efectiva.

En esta sección se abordarán los siguientes temas:

- Declaración de manejadores
- Tipos de manejadores
- Manejo condicional de errores
- Registro de errores

2.5. Manejo de errores (I)

En **MySQL**, el tratamiento de errores dentro de bloques se realiza mediante manejadores definidos explícitamente con la sentencia `DECLARE ... HANDLER`. MySQL permite capturar errores mediante los tipos `CONTINUE` y `EXIT`.

Sintaxis general

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    SET @error = 'Se produjo un error';
```

Captura de errores con manejo condicional

```
DELIMITER //
CREATE PROCEDURE ejemplo_error()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
        SELECT 'Se ha producido un error';

    DECLARE total INT;
    SET total = 100 / 0; -- Causa un error de división por cero
END;
//
DELIMITER ;
```

MySQL no utiliza `EXCEPTION` como PL/SQL, sino `DECLARE HANDLER`. No existen excepciones con nombres simbólicos predefinidos, sino que se capturan mediante condiciones como:

- `SQLEXCEPTION`
- `SQLWARNING`
- `NOT FOUND`

Tipos de manejadores

- **CONTINUE:** la ejecución continúa tras manejar el error
- **EXIT:** sale del bloque donde se produce el error

2.5.1. Manejo de errores (II)

Los manejadores tienen **ámbito local**, igual que las variables. Si se define un nuevo manejador en un subbloque con el mismo tipo que uno superior, **prevalece el local**.

MySQL no permite continuar automáticamente con la ejecución si se produce un error, a menos que el manejador lo gestione correctamente. Para ello, se recomienda encapsular instrucciones sensibles dentro de bloques manejados.

Ejemplo: ejecutar una instrucción crítica dentro de un bloque con control de errores

```
DELIMITER //
```

```
CREATE PROCEDURE inserta_agente()  
BEGIN  
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
    BEGIN  
        -- Intento alternativo o log del error  
        INSERT INTO errores (mensaje) VALUES ('Error al insertar familia');  
    END;  
  
    INSERT INTO familias VALUES (...);  
    INSERT INTO agentes VALUES (...);  
END;  
//  
DELIMITER ;
```

Para seguir ejecutando instrucciones después de un fallo, se recomienda usar **múltiples bloques BEGIN...END anidados**.

...

2.5.2. Manejo de errores (III)

Cuando se desea distinguir entre errores que surgen en distintas sentencias, se puede usar una **variable localizadora**:

```
DELIMITER //
```

```
CREATE PROCEDURE localizar_error()  
BEGIN  
    DECLARE sent INT DEFAULT 0;  
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
    BEGIN  
        IF sent = 0 THEN  
            INSERT INTO errores VALUES ('Fallo en consulta 1');  
        ELSEIF sent = 1 THEN
```

```

        INSERT INTO errores VALUES ('Fallo en consulta 2');
    END IF;
END;

SELECT * FROM agentes;
SET sent = 1;
SELECT * FROM familias;
END;
//
DELIMITER ;

```

Si no se captura el error en el bloque actual, este se **propaga al bloque exterior** (si existe).

MySQL no soporta RAISE explícito, pero se puede simular controlando el flujo y las variables asociadas.

2.5.3. Manejo de errores (IV)

En MySQL se puede registrar el **código y mensaje del error** usando GET DIAGNOSTICS, equivalente a SQLCODE y SQLERRM en Oracle.

Sintaxis:

```

DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
GET DIAGNOSTICS CONDITION 1
    @codigo = MYSQL_ERRNO, @mensaje = MESSAGE_TEXT;

```

Ejemplo completo:

```

DELIMITER //
CREATE PROCEDURE registrar_error()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    BEGIN
        GET DIAGNOSTICS CONDITION 1
            @cod = MYSQL_ERRNO, @msg = MESSAGE_TEXT;
        INSERT INTO errores (codigo, mensaje) VALUES (@cod, @msg);
    END;

    -- Error intencionado
    SET @x = 1 / 0;
END;
//
DELIMITER ;

```

MySQL no soporta `RAISE_APPLICATION_ERROR` ni `PRAGMA EXCEPTION_INIT`. Sin embargo, podemos simularlo registrando códigos personalizados manualmente.

■ **Consulta:** [MySQL Diagnostics](#)

Autoevaluación

2.5

- ✗ Todas las excepciones están predefinidas y no se pueden definir nuevas
- ✓ Las excepciones definidas por el usuario deben lanzarse explícitamente (mediante lógica propia)
- ✗ Es obligatorio declarar todas las excepciones predefinidas que vamos a usar

2.5.3

- ✓ Podemos lanzar nuestros propios mensajes de error a las aplicaciones (de forma simulada)
- ✗ No se puede acceder al mensaje del error generado
- ✓ Podemos asociar errores personalizados usando `GET DIAGNOSTICS` y registro manual

2.6. SQL embebido

El SQL embebido es una técnica que permite integrar sentencias SQL dentro de un lenguaje de programación, como MySQL. Esto permite realizar operaciones de bases de datos directamente desde el código, facilitando la manipulación y consulta de datos.

En esta sección se abordarán los siguientes temas:

- Uso de SQL en procedimientos y funciones
- SQL dinámico
- Limitaciones y consideraciones

2.6. Sentencias SQL en programas MySQL

En un bloque de procedimiento o función en **MySQL**, se utilizan sentencias SQL para recuperar y modificar los datos de las tablas de una base de datos.

Hasta ahora hemos trabajado con SQL de forma interactiva: escribíamos sentencias y obteníamos resultados directamente en consola. Para trabajar dentro de programas, empleamos **SQL embebido**, es decir, sentencias SQL integradas en bloques `BEGIN...END`, con resultados almacenados en variables para su posterior tratamiento.

MySQL permite usar plenamente el **Lenguaje de Manipulación de Datos (DML)**: SELECT, INSERT, UPDATE, DELETE, así como comandos de control de transacciones (START TRANSACTION, COMMIT, ROLLBACK).

Limitación: No se puede usar directamente el **Lenguaje de Definición de Datos (DDL)** (CREATE TABLE, ALTER TABLE, DROP TABLE) ni el **Lenguaje de Control de Datos (DCL)** (GRANT, REVOKE) dentro de procedimientos almacenados. Para ello, se utiliza **SQL dinámico** con PREPARE, EXECUTE y DEALLOCATE PREPARE.

Uso de SELECT en procedimientos

La instrucción SELECT debe devolver una sola fila. Si devuelve más de una, se produce el error: Row 1 was truncated. Si no devuelve ninguna, se puede capturar con un NOT FOUND HANDLER.

Sintaxis general:

```
SELECT columna1 [, columna2, ...] INTO var1 [, var2, ...]
FROM tabla
[WHERE condición];
```

- La cláusula INTO es obligatoria.
- El número de columnas en la SELECT debe coincidir con el número de variables.

Ejemplo:

```
DELIMITER //
CREATE PROCEDURE obtener_cliente()
BEGIN
    DECLARE nombre_cliente VARCHAR(50);

    SELECT nombre INTO nombre_cliente
    FROM clientes
    WHERE id_cliente = 10;

    SELECT CONCAT('Cliente: ', nombre_cliente);
END;
//
DELIMITER ;
```

Si el resultado devuelve más de una fila:

ERROR 1172 (42000): Result consisted of more than one row

Uso de INSERT, UPDATE, DELETE

Estas instrucciones se usan igual que en SQL estándar.

Ejemplo INSERT:

```
DELIMITER //
CREATE PROCEDURE insertar_producto()
BEGIN
    INSERT INTO productos (nombre, precio)
    VALUES ('Teclado mecánico', 89.95);
END;
//
DELIMITER ;
```

Ejemplo UPDATE:

```
DELIMITER //
CREATE PROCEDURE actualizar_precio()
BEGIN
    UPDATE productos
    SET precio = precio * 1.10
    WHERE categoria = 'tecnología';
END;
//
DELIMITER ;
```

Ejemplo DELETE:

```
DELIMITER //
CREATE PROCEDURE borrar_temporal()
BEGIN
    DELETE FROM logs_temporales
    WHERE fecha < CURDATE() - INTERVAL 30 DAY;
END;
//
DELIMITER ;
```

SQL dinámico: ejecutar DDL/DCL en MySQL

Para ejecutar CREATE, DROP, GRANT, etc., se debe usar SQL dinámico con PREPARE:

```
SET @query = 'CREATE TABLE temporal (id INT, nombre VARCHAR(50))';
PREPARE stmt FROM @query;
```

```
EXECUTE stmt;  
DEALLOCATE PREPARE stmt;
```

! **Importante:** No se puede ejecutar DDL directamente desde CREATE PROCEDURE. Se debe usar SQL dinámico como se muestra arriba.

Referencias:

[MySQL SELECT INTO documentation](#)

[MySQL Prepared Statements](#)

Ejercicios prácticos

Ejercicio 1:

Crea un procedimiento que reciba un `id_cliente` como parámetro y devuelva su nombre y email en variables.

```
DELIMITER //  
CREATE PROCEDURE datos_cliente(IN id INT, OUT nom VARCHAR(50), OUT correo VARCHAR(100))  
BEGIN  
    SELECT nombre, email INTO nom, correo  
    FROM clientes  
    WHERE id_cliente = id;  
END;  
//  
DELIMITER ;
```

Ejercicio 2:

Inserta un nuevo registro en una tabla ventas a partir de datos fijos.

Ejercicio 3:

Escribe un procedimiento que elimine productos cuya cantidad en stock sea 0 y devuelva el número de filas afectadas usando la variable `ROW_COUNT()`.

```
DELIMITER //  
CREATE PROCEDURE eliminar_sin_stock()  
BEGIN  
    DELETE FROM productos WHERE stock = 0;  
    SELECT ROW_COUNT() AS filas_eliminadas;  
END;
```

```
//  
DELIMITER ;
```

Ejercicio 4 (Avanzado):

Utiliza SQL dinámico para crear una tabla temporal cuyo nombre se pase como parámetro.

```
DELIMITER //  
CREATE PROCEDURE crear_tabla(IN nombre_tabla VARCHAR(50))  
BEGIN  
    SET @query = CONCAT('CREATE TABLE ', nombre_tabla, ' (id INT, dato VARCHAR(100))');  
    PREPARE stmt FROM @query;  
    EXECUTE stmt;  
    DEALLOCATE PREPARE stmt;  
END;  
//  
DELIMITER ;
```

■ Referencias:

- [MySQL SELECT INTO documentation](#)
- [MySQL Prepared Statements](#)

3. Tipos de datos compuestos

Caso práctico

María, con la ayuda de **Juan**, ha aprendido a manejar los tipos de datos simples. Juan le comenta que en la mayoría de los lenguajes de programación existen tipos de datos compuestos, que permiten modelar estructuras más complejas. María y Juan preguntan a **Ada** sobre este tema, quien les confirma que en PL/SQL —y en nuestro caso, **MySQL**— también existen formas de trabajar con estructuras compuestas, y los anima a estudiar este capítulo para dominarlas.

En este capítulo conoceremos cómo modelar estructuras complejas en **MySQL** mediante tipos de datos compuestos, equivalentes a los RECORD de PL/SQL, utilizando constructos como tipos definidos por el usuario, estructuras derivadas de tablas (%ROWTYPE en Oracle), y simulaciones mediante estructuras JSON o tablas temporales en MySQL.



"Todo lo complejo puede dividirse en partes simples" — *René Descartes*

3.1. Registros

MySQL no dispone de tipos RECORD como tal, pero podemos simularlos usando:

- Variables múltiples
- Estructuras SELECT INTO ... con varias columnas
- Tipos derivados de tablas con DECLARE var CURSOR FOR ... y FETCH INTO ...

Un registro es un grupo de variables relacionadas referenciadas por un nombre común. En MySQL, para simular este comportamiento se usan varias variables agrupadas por nombre.

Ejemplo básico con SELECT INTO múltiples variables

```
DELIMITER //
CREATE PROCEDURE obtener_direccion()
BEGIN
    DECLARE calle VARCHAR(50);
    DECLARE numero INT;
    DECLARE ciudad VARCHAR(30);

    SELECT dir_calle, dir_numero, dir_ciudad
    INTO calle, numero, ciudad
    FROM direcciones
    WHERE id = 1;

    SELECT CONCAT('Dirección: ', calle, ' ', numero, ', ', ciudad);
END;
//
DELIMITER ;
```

Simulación de %ROWTYPE usando cursores

```
DELIMITER //
CREATE PROCEDURE cursor_registro()
BEGIN
    DECLARE v_id INT;
    DECLARE v_nombre VARCHAR(40);
    DECLARE v_padre INT;
    DECLARE v_oficina INT;

    DECLARE cur CURSOR FOR
        SELECT identificador, nombre, padre, oficina FROM familias;

    OPEN cur;
    FETCH cur INTO v_id, v_nombre, v_padre, v_oficina;
```

```
CLOSE cur;  
END;  
//  
DELIMITER ;
```

Reglas importantes

- No se pueden comparar registros directamente (IF r1 = r2 es ilegal).
- No se pueden asignar estructuras completas si no son exactamente del mismo tipo.
- No se pueden usar registros directamente en INSERT ... VALUES, pero sí se puede descomponer campo a campo.
- Sí es legal hacer SELECT * INTO en variables individuales o en cursores.

Ejemplo de estructura que sería inválida en MySQL:

```
-- ILEGAL EN MYSQL:  
SET registro1 := registro2;  
INSERT INTO familias VALUES (registro1);
```

✓ En su lugar, se deben declarar variables independientes y usarlas explícitamente.

Ejercicio práctico propuesto

Objetivo: simular un registro de dirección con varias variables, recuperar los datos y mostrarlos formateados.

```
DELIMITER //  
CREATE PROCEDURE mostrar_direccion()  
BEGIN  
    DECLARE v_calle VARCHAR(50);  
    DECLARE v_numero INT;  
    DECLARE v_puerta VARCHAR(10);  
    DECLARE v_ciudad VARCHAR(30);  
  
    SELECT calle, numero, puerta, ciudad  
    INTO v_calle, v_numero, v_puerta, v_ciudad  
    FROM direcciones  
    WHERE id = 3;  
  
    SELECT CONCAT(v_calle, ' nº', v_numero, ', puerta ', v_puerta, ' (', v_ciudad, ' )'  
END;  
//  
DELIMITER ;
```

Extensión: crea un cursor que recorra todas las filas de la tabla familias y muestre cada fila como si fuera un registro.

■ Referencias:

- [MySQL SELECT INTO syntax](#)
- [MySQL cursors](#)

3.3. Cursores

Un **cursor** en MySQL permite recorrer fila a fila el resultado de una consulta. Existen solo **cursores explícitos** (no implícitos como en Oracle). Los cursores nos permiten procesar de forma iterativa los resultados de una SELECT que devuelve varias filas.

Declaración de cursor

```
DECLARE nombre_cursor CURSOR FOR SELECT ...;
```

Atributos

MySQL no tiene atributos como %FOUND o %NOTFOUND, pero se pueden simular con:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND ...;
```

Ejemplo básico:

```
DELIMITER //
```

```
CREATE PROCEDURE ejemplo_cursor()  
BEGIN  
    DECLARE fin INT DEFAULT 0;  
    DECLARE id INT;  
    DECLARE nombre VARCHAR(50);  
  
    DECLARE cur CURSOR FOR SELECT id, nombre FROM empleados;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;  
  
    OPEN cur;  
    bucle: LOOP  
        FETCH cur INTO id, nombre;  
        IF fin = 1 THEN  
            LEAVE bucle;  
        END IF;
```

```

        SELECT CONCAT('Empleado: ', nombre);
    END LOOP;
    CLOSE cur;
END;
//
DELIMITER ;

```

3.3.1. Cursores explícitos

MySQL permite definir cursores, pero **no permite parámetros directamente en la declaración** (a diferencia de Oracle). Todo el control se realiza dentro del procedimiento, utilizando variables.

Bucle implícito tipo FOR (simulado)

```

DELIMITER //
CREATE PROCEDURE recorrer_familias()
BEGIN
    DECLARE fin INT DEFAULT FALSE;
    DECLARE id INT;
    DECLARE nombre VARCHAR(50);

    DECLARE cur CURSOR FOR SELECT identificador, nombre FROM familias;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;

    OPEN cur;
    REPEAT
        FETCH cur INTO id, nombre;
        IF NOT fin THEN
            SELECT CONCAT('Familia: ', nombre);
        END IF;
    UNTIL fin END REPEAT;
    CLOSE cur;
END;
//
DELIMITER ;

```

Autoevaluación

En MySQL los cursores son abiertos al definirlos.

-  Verdadero

- ☒ Falso

>

3.3.2. Cursores variables

MySQL **no tiene cursores variables** (REF CURSOR) como Oracle. Sin embargo, se puede lograr un comportamiento similar mediante el uso de **SQL dinámico** con PREPARE, EXECUTE, y estructuras auxiliares.

Un cursor variable en Oracle es como un puntero a cualquier consulta. En MySQL, simulamos esto construyendo la consulta como una cadena de texto y ejecutándola dinámicamente.

Simulación de cursores variables en MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE cursor_variable_simulado(IN oficina_id INT)
```

```
BEGIN
```

```
    SET @query = CONCAT('SELECT nombre FROM agentes WHERE oficina = ', oficina_id);
```

```
    PREPARE cur FROM @query;
```

```
    EXECUTE cur;
```

```
    DEALLOCATE PREPARE cur;
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Podemos cambiar dinámicamente la consulta con nuevos parámetros sin necesidad de cerrar ningún cursor.

 **Consulta:** [MySQL PREPARE](#)

Autoevaluación

- ☒ A los cursores variables no podemos pasarles parámetros al abrirlos (Verdadero tanto en Oracle como en su simulación en MySQL)
- ☒ Los cursores variables se abren exactamente igual que los cursores explícitos

4. Abstracción en MySQL

En este bloque abordamos cómo organizar y reutilizar el código dentro de **MySQL** utilizando **funciones**, **procedimientos** y el concepto de **modularidad**. Esto nos permite aplicar diferentes niveles de abstracción: saber *qué* hace algo, sin preocuparse de *cómo* lo hace.

MySQL permite:

- Definir procedimientos almacenados (CREATE PROCEDURE)
- Definir funciones almacenadas (CREATE FUNCTION)
- *No permite* agrupar funciones o procedimientos en paquetes como en Oracle.

4.1. Subprogramas

Un **subprograma** en MySQL es un bloque reutilizable de código SQL definido en el servidor. Existen dos tipos:

- **Procedimientos:** no devuelven valor directamente
- **Funciones:** devuelven un valor mediante RETURN

Sintaxis de una función

```
DELIMITER //
CREATE FUNCTION nombre_funcion(param1 INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE resultado INT;
    SET resultado = param1 * 2;
    RETURN resultado;
END;
//
DELIMITER ;
```

Sintaxis de un procedimiento

```
DELIMITER //
CREATE PROCEDURE nombre_procedimiento(IN param1 INT, OUT resultado INT)
BEGIN
    SET resultado = param1 * 2;
END;
//
DELIMITER ;
```

Consideraciones importantes

- Una función debe devolver siempre un valor.
- No se pueden imponer restricciones NOT NULL en parámetros.
- Los subprogramas se deben declarar antes de invocarse (si están en el mismo bloque).
- Para hacer subprogramas mutuamente recursivos en MySQL, se deben almacenar por separado y luego llamarse entre sí.

Ejemplo de definición anidada con llamada a función

```
DELIMITER //
CREATE FUNCTION hijos_familia(id_familia INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM agentes WHERE familia = id_familia;
    RETURN total;
END;
//
DELIMITER ;
```

Autoevaluación

-  **Una función siempre debe devolver un valor.** — Verdadero
-  **En MySQL no podemos definir subprogramas mutuamente recursivos.** — Falso

4.1.1. Almacenar subprogramas en la base de datos

En **MySQL**, para almacenar un subprograma (función o procedimiento) en la base de datos, se utiliza la sintaxis `CREATE PROCEDURE` o `CREATE FUNCTION`. Aunque MySQL no tiene `CREATE OR REPLACE`, puedes usar `DROP PROCEDURE IF EXISTS` o `DROP FUNCTION IF EXISTS` antes de volver a crearlos.

Ejemplo:

```
DROP FUNCTION IF EXISTS hijos_familia;
DELIMITER //
CREATE FUNCTION hijos_familia(id_familia INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE hijos INT;
```

```
SELECT COUNT(*) INTO hijos FROM agentes WHERE familia = id_familia;
RETURN hijos;

END;
//
DELIMITER ;
```

Para invocar un subprograma:

```
CALL nombre_procedimiento(...);
SELECT nombre_funcion(...);
```

MySQL permite consultar la información de subprogramas almacenados mediante:

- SHOW CREATE PROCEDURE nombre
- SHOW CREATE FUNCTION nombre
- INFORMATION_SCHEMA.ROUTINES

Autoevaluación

Una vez que hemos almacenado un subprograma en la base de datos podemos consultar su código mediante la vista USER_OBJECTS.

-  Verdadero
-  Falso (en MySQL usamos INFORMATION_SCHEMA.ROUTINES)

4.1.2. Parámetros de los subprogramas

En **MySQL**, los subprogramas pueden tener parámetros de tipo:

- IN: entrada (por defecto)
- OUT: salida
- INOUT: entrada/salida

Ejemplo con los tres tipos:

```
DELIMITER //
CREATE PROCEDURE ejemplo_parametros(IN entrada INT, OUT salida INT, INOUT suma INT)
BEGIN
    SET salida = entrada * 2;
    SET suma = suma + entrada;
```

```
END;  
//  
DELIMITER ;
```

Los parámetros se pueden pasar por **posición** (orden). MySQL **no permite notación por nombre directamente**, aunque puede simularse en lógica de aplicación o mediante interfaces.

Autoevaluación

- ☒ En MySQL podemos usar la notación posicional para pasar parámetros.
- ☒ No existen los parámetros de salida ya que para eso existen las funciones.
- ☒ Los parámetros de entrada los podemos inicializar a un valor por defecto (en versiones recientes de MySQL).

5. Disparadores

Caso práctico

Ilustración en la que se representa un chico rodeado de muchas columnas y una tira de papel con algún texto se extiende por entre todas las columnas. (ITE - Uso educativo no comercial)

Juan y María ya han hecho un paquete en el que tienen agrupadas las funciones más usuales que realizan sobre la base de datos de juegos on-line. Sin embargo, María ve que hay cosas que aún no pueden controlar. Por ejemplo, María quiere que la clave y el usuario de un jugador o jugadora no puedan ser la misma y eso no sabe cómo hacerlo. Juan le dice que para ese cometido están los **disparadores** y sin más dilaciones se pone a explicarle a María para qué sirven y cómo utilizarlos.

En este apartado vamos a tratar una herramienta muy potente e importante proporcionada por PL/SQL para programar nuestra base de datos y mantener la integridad y seguridad: los **disparadores** o *triggers* en inglés.

¿Qué es un disparador?

Un **disparador** no es más que un procedimiento que se ejecuta automáticamente cuando se realiza alguna sentencia sobre la base de datos o sus tablas, y bajo unas circunstancias específicas establecidas en su definición.

Tipos de disparadores

- **De tablas:** los más comunes. Se ejecutan al realizar INSERT, UPDATE o DELETE sobre una tabla.
- **De sustitución:** conocidos como INSTEAD OF. Se asocian solo a **vistas**.

- **De sistema:** se ejecutan ante eventos del sistema como LOGON, LOGOFF, CREATE, DROP, etc.

Antes o después de la operación

Un disparador puede lanzarse **BEFORE** o **AFTER** de la operación que lo activa.

Disparadores de tabla

Previenen transacciones erróneas o automatizan tareas. Pueden lanzarse:

- Al insertar, actualizar o eliminar.
- **Una vez por sentencia:** FOR STATEMENT
- **Una vez por cada fila afectada:** FOR EACH ROW

Usos típicos de los disparadores

- Auditoría de cambios en los datos
- Aplicar reglas de integridad personalizadas
- Generar automáticamente valores derivados

Recomendaciones de diseño

- Evitar duplicar funciones que ya puede hacer el SGBD
- Evitar disparadores excesivamente largos; usar subprogramas auxiliares
- Prevenir recursividad involuntaria en los disparadores

Autoevaluación

- ❌ En PL/SQL sólo podemos definir disparadores de fila. — **Falso**
- ✅ La diferencia entre un disparador de fila y uno de sentencia es que el de fila se lanza una vez por fila afectada, mientras que el de sentencia se lanza una sola vez. — **Verdadero**

5.1. Definición de disparadores de tablas

Ilustración en la que se ven dos chicos, un cañón y se puede leer "Bang!". (ITE - Uso educativo no comercial)

De los tres tipos de disparadores vistos, en MySQL nos centraremos en los **disparadores asociados a tablas**, ya que MySQL solo permite definir **disparadores de fila** (no de sistema ni de sustitución).

Conceptos clave

Un disparador en MySQL se asocia a una tabla concreta y se ejecuta **antes** (BEFORE) o **después** (AFTER) de una operación INSERT, UPDATE o DELETE.

Sintaxis general en MySQL:

```
CREATE TRIGGER nombre_trigger
  {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
  ON nombre_tabla
  FOR EACH ROW
  BEGIN
    -- cuerpo del trigger
  END;
```

MySQL no soporta REFERENCING ni WHEN. Los valores OLD.columna y NEW.columna se utilizan directamente dentro del cuerpo del trigger para acceder a los valores anteriores o nuevos de una fila.

Ejemplo:

```
CREATE TRIGGER validar_usuario
  BEFORE INSERT ON jugadores
  FOR EACH ROW
  BEGIN
    IF NEW.usuario = NEW.clave THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'La clave no puede ser igual al usuario.';
    END IF;
  END;
```

Restricciones importantes en MySQL

- No se puede acceder a la **misma tabla** que lanza el trigger dentro de su cuerpo (se considera una tabla "mutante").
- Un trigger **no puede realizar operaciones sobre la misma tabla** que lo activa.
- No se permite definir más de **un trigger con la misma combinación** de momento (BEFORE/AFTER) y evento (INSERT/UPDATE/DELETE) para una misma tabla en MySQL 5.x (aunque MySQL 8.0 ya lo permite con ciertas condiciones).

Ejemplo inválido en MySQL (causaría error):

```
CREATE TRIGGER trigger_invalido
  BEFORE UPDATE ON agentes
  FOR EACH ROW
  BEGIN
    -- ❌ Esto lanzaría un error
```

```
SELECT identificador FROM agentes WHERE oficina = 1;  
END;
```

Orden de ejecución de disparadores

En MySQL 8.0 se puede especificar el `FOLLOWS` y `PRECEDES` para controlar el orden, pero por defecto el orden es indeterminado si hay más de un trigger del mismo tipo.

Información de disparadores

En MySQL, la información sobre los disparadores se puede consultar mediante:

- `SHOW TRIGGERS;`
- `SELECT * FROM INFORMATION_SCHEMA.TRIGGERS;`

Ejemplo de consulta:

```
SHOW TRIGGERS FROM nombre_base_datos;  
SELECT TRIGGER_NAME, EVENT_MANIPULATION, ACTION_TIMING  
FROM INFORMATION_SCHEMA.TRIGGERS  
WHERE TRIGGER_SCHEMA = 'nombre_base_datos';
```

Ejemplo práctico con disparadores en MySQL

Objetivo: Evitar que un jugador pueda registrarse o modificarse con una clave igual a su nombre de usuario.

1. Crear tabla jugadores

```
CREATE TABLE jugadores (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  usuario VARCHAR(50) NOT NULL,  
  clave VARCHAR(50) NOT NULL  
);
```

2. Trigger BEFORE INSERT

```
DELIMITER //  
CREATE TRIGGER trig_evitar_usuario_igual_clave_insert  
BEFORE INSERT ON jugadores  
FOR EACH ROW  
BEGIN  
  IF NEW.usuario = NEW.clave THEN  
    SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'ERROR: La clave no puede ser igual al usuario.';
    END IF;
END;
//
DELIMITER ;
```

3. Trigger BEFORE UPDATE

```
DELIMITER //
CREATE TRIGGER trig_evitar_usuario_igual_clave_update
BEFORE UPDATE ON jugadores
FOR EACH ROW
BEGIN
    IF NEW.usuario = NEW.clave THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: La clave no puede ser igual al usuario.';
    END IF;
END;
//
DELIMITER ;
```

4. Pruebas de inserción

```
-- Inserción válida
INSERT INTO jugadores (usuario, clave) VALUES ('luis2024', 'claveSegura');

-- Inserción inválida (activará el trigger y lanzará error)
INSERT INTO jugadores (usuario, clave) VALUES ('admin', 'admin');
```

5. Prueba de actualización

```
-- Intento de modificar la clave a igual que el usuario
UPDATE jugadores SET clave = 'luis2024' WHERE id = 1;
```

6. Verificación de los disparadores

```
SHOW TRIGGERS LIKE 'jugadores';
```

 **Recomendación:** Usar `SIGNAL SQLSTATE '45000'` permite personalizar el mensaje de error mostrado cuando el disparador detecta una condición inválida.



Ejercicios de repaso – Unidad 6: Programación en MySQL

A continuación se proponen ejercicios ordenados de menor a mayor dificultad para afianzar los conceptos tratados en esta unidad.

◆ Nivel 1 – Básico

1. Crea un procedimiento que reciba dos números y devuelva su suma en un parámetro OUT.
2. Define una función que reciba un número y devuelva su cuadrado.
3. Crea una variable de sesión con @ y almacena en ella tu nombre. Luego realiza una consulta que la concatene con un saludo.
4. Crea una tabla llamada empleados con campos id, nombre, departamento y realiza una inserción mediante un procedimiento.

◆ Nivel 2 – Intermedio

5. Crea un procedimiento que reciba un id de empleado y devuelva su nombre y departamento.
6. Simula el uso de registros: realiza una `SELECT INTO` de varios campos de la tabla clientes en variables individuales.
7. Define un cursor que recorra todos los empleados e imprima uno a uno su nombre y departamento.
8. Crea una función recursiva que calcule el factorial de un número.

◆ Nivel 3 – Avanzado

9. Crea un **disparador** que impida insertar un producto cuyo precio sea menor o igual a 0.
10. Implementa una tabla de `historial_ventas` y un **trigger AFTER INSERT** que guarde cada venta insertada en ventas.
11. Define una estructura con control de errores usando `DECLARE HANDLER` que capture un error al dividir entre 0 y registre un mensaje en una tabla de errores.
12. Crea un procedimiento que utilice SQL dinámico para crear una tabla cuyo nombre se pase como parámetro.



Opcional: Desafío final

Desarrolla una pequeña aplicación que:

- Gestione empleados, departamentos y tareas.
- Use procedimientos para insertar datos, funciones para cálculos de productividad, cursores para listar tareas pendientes y disparadores para evitar datos inconsistentes.



Repaso global – Unidad 6: Programación en MySQL

Esta unidad nos ha introducido en el **lenguaje de programación de bases de datos en MySQL**, utilizando SQL/PSM (Persistent Stored Modules). A lo largo de los distintos bloques hemos aprendido a:

- Comprender la estructura sintáctica de los programas: delimitadores, literales, identificadores y comentarios.
- Declarar **variables** y **constantes** dentro de procedimientos y funciones.
- Utilizar **estructuras de control**: condicionales (IF, CASE) e iterativas (WHILE, LOOP).
- Manejar errores mediante DECLARE HANDLER y GET DIAGNOSTICS.
- Insertar sentencias SQL dentro de bloques, incluyendo el uso de **SQL dinámico** con PREPARE.
- Simular **tipos compuestos** como registros y recorrer conjuntos de resultados con **cursores**.
- Crear **procedimientos** y **funciones almacenadas** que encapsulen lógica de negocio reutilizable.
- Definir **disparadores** (*triggers*) para validar, automatizar y auditar acciones sobre las tablas.
- Conocer la existencia de **APIs externas** para conectar aplicaciones en lenguajes como Python, PHP o Java a nuestra base de datos.

El dominio de estas técnicas permite integrar lógica avanzada directamente en el servidor de base de datos, simplificando el desarrollo de aplicaciones robustas, seguras y mantenibles.

 **Consejo:** Practica todos los ejemplos en un entorno real como MySQL Workbench y asegúrate de comprender no solo cómo se escribe el código, sino *por qué* se estructura de esa manera.



Caso de estudio: Agencia de Agentes

Este caso se basa en una agencia de representación con múltiples **oficinas**, **familias jerárquicas** y una estructura de **agentes** organizados por especialización, supervisión y localización.

La base de datos está compuesta por tres tablas principales:

- **oficinas**: contiene información de cada sede física (nombre, dirección, localidad...)
- **familias**: representa unidades organizativas jerárquicas (pueden depender unas de otras o de oficinas)
- **agentes**: personas que trabajan en la agencia y que pueden estar asociadas a una familia o a una oficina



Objetivos pedagógicos

- Practicar la creación de tablas con claves primarias y foráneas
- Insertar datos jerárquicos y relacionales
- Realizar consultas con JOIN y GROUP BY sobre estructuras organizativas
- Implementar procedimientos, funciones y disparadores sobre un escenario realista



Estructura general del modelo

- Cada familia puede estar asignada a una oficina o depender de otra familia
- Un agente puede estar vinculado a una familia o directamente a una oficina
- Se permite representar jerarquías de hasta 3 o 4 niveles en familias
- Los agentes tienen atributos como habilidad, categoría, nombre de usuario y contraseña



Descarga del script SQL

Para comenzar a trabajar con este caso, descarga y ejecuta el siguiente script en tu entorno MySQL:



Descargar CreaCasoEstudio_MySQL_Completo.sql



Ejemplos de futuras actividades prácticas

- Obtener el número de agentes por familia
- Detectar agentes sin familia asignada
- Crear un procedimiento que liste los agentes por oficina
- Definir un disparador que registre movimientos entre oficinas
- Simular un informe de supervisores por localidad

✂ Este caso será la base para los ejercicios de programación con procedimientos, funciones, cursores y triggers que desarrollaremos a lo largo de la unidad.